GEP Phase 2 UI 개발지시서 v1.0

문서코드: GEP-UI-DEV-001
작성일: 2026년 2월 21일
작성자: 디자인팀장
수신: 노팀장 → 고팀장
승인: 조대표님
상태: ✅ 설계 완료 / 개발 지시 대기

---

1. 개요

목표

ACIU에서 검증된 통계 UX를 GEP 구조에 맞게 이식한다.
사용자가 자신의 학습 상태를 한눈에 파악하고 합격 전략을 세울 수 있는 화면 구성.

개발 단계

단계	내용	우선순위
UI-1단계	데이터 구조 (localStorage 스키마)	최우선
UI-2단계	문제화면 내 금일/누적 통계	1순위
UI-3단계	대문 통계 대시보드	2순위

핵심 원칙

- **선택은 회차로, 통계는 과목으로** (소분류 12개 세부과목 기준)
- 기존 Phase 1 코드 최소 수정 (신규 파일 추가 방식)
- 레고블록 철학 — StatsPanel은 UI-2/UI-3 공통 재활용
- 모바일 우선 설계

---

2. GEP 분류 체계 (확정)

구분	분류	내용
대분류	회차	23, 24, 25, 26, 27, 28, 29, 30, 31회

구분	분류	내용
중분류	부 단위	법령 / 손보1부 / 손보2부
소분류	세부과목	보험업법, 상법, 위험관리, 세제재무, 자동차보험, 특종보험, 보증보험, 연금저축, 화재보험, 해상보험, 항공우주, 재보험

총 문제수: 1,080문제 (9회차 × 120문제)
통계 집계 기준: 소분류(세부과목) 12개

---

3. UI-1단계: 데이터 구조 설계

3-1. localStorage 키 체계

키	용도	충돌 여부
`gep_user`	사용자 정보	신규 (충돌 없음)
`gep_stats_v1`	통계 데이터 (핵심)	신규 (충돌 없음)
`gep_progress_v1`	진도 데이터	신규 (기존 `gep_progress`와 분리)
`gep_history`	기존 Phase 1	유지 (수정 금지)
`gep_settings`	기존 Phase 1	유지 (수정 금지)
`gep_last`	기존 Phase 1	유지 (수정 금지)

3-2. gep_user 스키마

```json
{
  "name": "조대표님",
  "phone": "010-xxxx-xxxx",
  "examDate": "2026-05-15",
  "registeredAt": "2026-02-21",
  "schemaVersion": 1
}
```

3-3. gep_stats_v1 스키마 (핵심)

```json
```

```
{
  "global": {
    "cumulative": { "attempted": 0, "correct": 0 },
    "daily": {
      "2026-02-21": { "attempted": 0, "correct": 0 }
    }
  },
  "subjects": {
    "보험업법":   { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "상법":       { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "위험관리":   { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "세제재무":   { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "자동차보험": { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "특종보험":   { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "보증보험":   { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "연금저축":   { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "화재보험":   { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "해상보험":   { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "항공우주":   { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "재보험":     { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} }
  },
  "rounds": {
    "23": { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "24": { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "25": { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "26": { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "27": { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "28": { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "29": { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "30": { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} },
    "31": { "cumulative": { "attempted": 0, "correct": 0 }, "daily": {} }
  }
}
```


3-4. gep_progress_v1 스키마

```
json
```

```
{
  "last": {
    "round": "30",
    "subject": "손보1부",
    "subSubject": "자동차보험",
    "index": 14
  },
  "positions": {
    "30_손보1부_자동차보험": 14,
    "29_법령_보험업법": 40
  }
}
```

3-5. 핵심 로직 규칙

정답률 계산:

```javascript
// 0나누기 방지 필수
const accuracy = attempted === 0 ? 0 : Math.round((correct / attempted) * 100);
```

금일 키 생성:

```javascript
const getTodayKey = () => new Date().toISOString().split('T')[0]; // "2026-02-21"
```

daily 키 없을 때 자동 생성:

```javascript
if (!stats.global.daily[today]) {
  stats.global.daily[today] = { attempted: 0, correct: 0 };
}
```

버전 체크 (앱 시작 시):

```javascript
const stored = JSON.parse(localStorage.getItem('gep_stats_v1'));
if (!stored || stored.schemaVersion !== CURRENT_VERSION) {
  // 마이그레이션 또는 초기화
}
```

---

4. UI-2단계: 문제화면 내 통계

4-1. 화면 레이아웃

정답 확인 전:

```
┌──────────────────────────────────────────────┐
│ [←홈] 30회/자동차보험  15/40  │ ← Header
├──────────────────────────────────────────────┤
│ ▰▰▰▰▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱▱▱▱▱ 37%      │ ← ProgressBar
├──────────────────────────────────────────────┤
│ [30회] [손보1부] [자동차보험]  │ ← 브레드크럼
│                        │
│  문제 텍스트 영역          │ ← QuestionView
│  (스크롤)                 │
│                        │
│  ┌──────────── 📊 통계 보기 ▼ ────────────┐  │ ← StatsPanel (접힘)
│  └──────────────────────────────────────┘  │
├──────────────────────────────────────────────┤
│  [이전]  [정답확인]  [다음]  │ ← 하단 고정
└──────────────────────────────────────────────┘
```

정답 확인 후 (통계 자동 펼침):

```
┌──────────────────────────────────────────────┐
│ [←홈] 30회/자동차보험  15/40  │
├──────────────────────────────────────────────┤
│ ▰▰▰▰▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱▱▱▱▱ 37%      │
├──────────────────────────────────────────────┤
│  ✅ 정답입니다! (또는 ❌ 오답)  │
│                        │
│  ┌──────────── 자동차보험 통계 ────────────┐  │
│  │ 📊 누적현황              │ │
│  │  푼문제  정답   정답률     │ │
│  │   125    98     78%      │ │
│  ├──────────────────────────────────────┤ │
│  │ 📅 금일현황              │ │
│  │  푼문제  정답   정답률     │ │
│  │   15     12     80%      │ │
│  └──────────────────────────────────────┘  │
├──────────────────────────────────────────────┤
│  [이전]  [정답확인]  [다음]  │
└──────────────────────────────────────────────┘
```

4-2. 신규 파일 목록

파일	구분	역할
src/stores/statsStore.js	신규	통계 상태관리 (Zustand)
src/components/StatsPanel.jsx	신규	통계 표시 컴포넌트
src/utils/statsStorage.js	신규	localStorage 읽기/쓰기
src/pages/Question.jsx	수정	statsStore 연동 추가

4-3. StatsPanel 컴포넌트 명세

```jsx
// src/components/StatsPanel.jsx
<StatsPanel
  subSubject="자동차보험"   // 현재 세부과목
  isVisible={isAnswered}    // 정답 확인 후 true
  stats={{
    cumulative: { attempted: 125, correct: 98 },
    daily:      { attempted: 15,  correct: 12 }
  }}
/>
```

4-4. statsStore 핵심 메서드

```javascript
```

```
// src/stores/statsStore.js (Zustand)
{
  // 통계 업데이트 (문제풀이 시 호출)
  updateStats: (subSubject, round, isCorrect) => {
    // global.cumulative 업데이트
    // global.daily[오늘] 업데이트
    // subjects[subSubject].cumulative 업데이트
    // subjects[subSubject].daily[오늘] 업데이트
    // rounds[round].cumulative 업데이트
    // localStorage 저장
  },

  // 특정 과목 통계 조회
  getStats: (subSubject) => { ... },

  // 전체 통계 조회 (대문용)
  getGlobalStats: () => { ... },

  // localStorage에서 로드
  loadFromStorage: () => { ... },

  // 전체 초기화
  reset: () => { ... }
}
```

4-5. 데이터 흐름

```
정답확인 버튼 클릭
  → examStore.submitAnswer() 실행
  → 중복 체크: userAnswers[questionId] 있으면 skip
  → statsStore.updateStats(subSubject, round, isCorrect)
  → gep_stats_v1 localStorage 저장
  → StatsPanel isVisible=true → 자동 펼침
  → 최신 수치 즉시 렌더링
```

4-6. 중복 기록 방지 (중요)

```javascript
// 이미 답한 문제로 이전 이동 후 재확인 시 중복 방지
const isAlreadyAnswered = !!userAnswers[question.id];
if (!isAlreadyAnswered) {
  statsStore.updateStats(subSubject, round, isCorrect);
}
```

4-7. 모바일 레이아웃 주의사항

```
css

/* StatsPanel이 하단 고정 버튼에 가려지지 않도록 */
.question-content {
  padding-bottom: 200px; /* 통계패널(120px) + 버튼(60px) + 여유 */
}

/* 하단 버튼 고정 */
.bottom-nav {
  position: fixed;
  bottom: 0;
  left: 0;
  right: 0;
  padding-bottom: env(safe-area-inset-bottom);
  z-index: 100;
}
```

---

5. UI-3단계: 대문 통계 대시보드

5-1. 화면 레이아웃

```
┌──────────────────────────────────────────────────────────┐
│ GEP 보험중개사        [⚙설정]│ ← Header
├──────────────────────────────────────────────────────────┤
│ 조대표님                  │
│ 시험까지    D-83일          │ ← DdayBanner
│ 2026년 5월 15일             │
├──────────────────────────────────────────────────────────┤
│ 📦 보유 문제수              │
│  ┌──────────┬──────────────┬──────────────────┐  │
│  │ 법령  │ 손보1부│  손보2부  │  │ ← QuestionStock
│  │ 360   │  360  │   360    │  │  (고정값 표시)
│  │       │       │ 합계 1,080│  │
│  └──────────┴──────────────┴──────────────────┘  │
├──────────────────────────────────────────────────────────┤
│ 📈 학습 진도 현황           │
│  ┌──────────┬──────────────┬──────────────────┐  │
│  │ 완료 │ 진도율 │  정답률    │  │ ← ProgressStats
│  │ 245  │  22%  │   76%     │  │
│  └──────────┴──────────────┴──────────────────┘  │
├──────────────────────────────────────────────────────────┤
│ 📅 금일 학습현황            │
│  ┌──────────┬──────────────┬──────────────────┐  │
```

```
│  │ 오늘 │ 오늘  │ 오늘     │  │ ← TodayStats
│  │  20  │  15  │  75%     │  │
│  │문제  │정답  │정답률    │  │
│  └──────┴──────┴──────────┘  │
├──────────────────────────────────────────────┤
│ 📊 과목별 현황                │
│  ┌──────────────────────────┐  │
│  │ 법령 ▼                  │  │ ← SubjectStats
│  │ 보험업법 ▰▰▰▰▱▱▱▱ 68%   │  │   (접이식, 부별)
│  │ 상법     ▰▰▰▰▰▰▰▰▰ 82%  │  │
│  │ 위험관리 ▰▰▰▱▱▱▱▱▱ 55% ⚠ │  │  ← 40% 미달 경고
│  │ 세제재무 ▰▰▰▰▱▱▱▱ 71%   │  │
│  ├──────────────────────────┤  │
│  │ 손보1부 ▶ (접힘)         │  │
│  ├──────────────────────────┤  │
│  │ 손보2부 ▶ (접힘)         │  │
│  └──────────────────────────┘  │
├──────────────────────────────────────────────┤
│ 🚀 이어풀기                   │
│  ┌──────────────────────────┐  │ ← ResumeCard (재활용)
│  │ 30회/자동차보험  15번부터 │  │
│  │    [바로 이어하기 →]      │  │
│  └──────────────────────────┘  │
└──────────────────────────────────────────────┘
```

5-2. 신규 파일 목록

파일	구분	역할
`src/components/DdayBanner.jsx`	신규	D-day + 사용자명 표시
`src/components/ProgressStats.jsx`	신규	누적 진도/정답률 3수치
`src/components/TodayStats.jsx`	신규	금일 통계 3수치
`src/components/SubjectStats.jsx`	신규	12과목 정답률 바 (접이식)
`src/pages/Home.jsx`	수정	statsStore 연동 추가

재활용 컴포넌트 (수정 금지):

- `ResumeCard.jsx` — 기존 그대로 사용
- `Header.jsx` — 기존 그대로 사용
- `StatsPanel.jsx` — UI-2에서 개발한 것 그대로 사용

5-3. DdayBanner 명세

```jsx
<DdayBanner
  userName="조대표님"
  examDate="2026-05-15"
/>
// D-day 계산: Math.ceil((examDate - today) / 86400000)
// 음수일 경우 "시험 종료" 표시
// 0이면 "D-Day!" 표시
```

5-4. SubjectStats 색상 규칙

정답률	색상	의미
60% 이상	초록	양호
40~60%	노랑	주의
40% 미만	빨강 + ⚠️	위험 (합격선 미달)
데이터 없음	회색	"-" 표시

5-5. 데이터 흐름

```
Home.jsx 마운트
  → statsStore.loadFromStorage()
  → gep_stats_v1 로드
  → gep_user 로드 (이름, 시험일)
  → 각 컴포넌트에 props 전달

진도율: global.cumulative.attempted / 1080 × 100
정답률: global.cumulative.correct / global.cumulative.attempted × 100
금일:   global.daily[getTodayKey()]
과목별: subjects[과목명].cumulative

문제풀기 후 복귀:
  → statsStore 이미 업데이트 완료
  → Home.jsx 재마운트 시 최신값 자동 반영
```

---

6. 전체 연동 구조

```
gep_stats_v1 (localStorage)
      ↕ 읽기/쓰기
 statsStore.js (Zustand)
      ↕ 구독
┌──────────────────────────┬──────────────────────────┐
│  Question.jsx    │   Home.jsx       │
│  (UI-2)          │   (UI-3)         │
│  StatsPanel      │   DdayBanner     │
│  - 누적현황      │   ProgressStats  │
│  - 금일현황      │   TodayStats     │
│  (과목별 표시)   │   SubjectStats   │
│                  │   ResumeCard     │
└──────────────────────────┴──────────────────────────┘
```

---

7. 개발 우선순위 및 순서

Step 1 — statsStore + statsStorage (기반 작업)

- `src/utils/statsStorage.js` 작성
- `src/stores/statsStore.js` 작성
- 단위 테스트: updateStats → localStorage 저장 확인

Step 2 — StatsPanel + Question.jsx 연동 (UI-2)

- `src/components/StatsPanel.jsx` 작성
- `src/pages/Question.jsx` 수정 (statsStore 연동)
- 테스트: 문제풀기 → 통계 즉시 반영 확인

Step 3 — 대문 컴포넌트 (UI-3)

- `DdayBanner`, `ProgressStats`, `TodayStats`, `SubjectStats` 작성
- `src/pages/Home.jsx` 수정
- 테스트: 대문 통계 정확성 + 과목별 색상 확인

Step 4 — 통합 테스트

- UI-2 → UI-3 데이터 정합성 확인
- 모바일 레이아웃 확인
- 자정 날짜 리셋 시뮬레이션

8. 테스트 시나리오 (고팀장 필수 확인)

시나리오	내용	확인 방법
T-01	신규 사용자 첫 접속	모든 수치 0, 이어풀기 카드 미표시
T-02	정답 시 통계 즉시 반영	정답확인 클릭 → 수치 +1 확인
T-03	오답 시 정답수 미증가	attempted +1, correct 유지
T-04	이전 문제 이동 후 재확인	중복 기록 없음 확인
T-05	과목 전환 시 해당 과목 통계만 표시	자동차보험 → 특종보험 전환
T-06	앱 재시작 후 통계 유지	localStorage 로드 정확성
T-07	금일 수치 자정 리셋	날짜 변경 시 금일 0으로 초기화
T-08	D-day 음수 처리	시험일 지난 경우 "시험 종료" 표시
T-09	합격선 40% 미달 경고	빨간색 + ⚠️ 표시
T-10	모바일 레이아웃	통계 패널 버튼 겹침 없음

9. 주의사항 (고팀장 필독)

기존 파일 수정 범위 최소화

- `Question.jsx`, `Home.jsx` 수정 시 기존 로직 건드리지 말 것
- statsStore 연동 코드만 추가

0나누기 방지 필수

- 모든 정답률 계산 시 attempted === 0 체크 필수

daily 키 자동 생성

- getTodayKey() 기준으로 없으면 자동 생성 후 업데이트

gep_stats_v1 키 고정

- 절대 `gep_stats` (버전 없는 키) 사용 금지 → 향후 마이그레이션 불가

10. 완료 기준

- [ ] 문제풀기 후 StatsPanel에 금일/누적 즉시 표시
- [ ] 과목 전환 시 해당 과목 통계만 표시
- [ ] 대문 진도율/금일현황 정확히 표시
- [ ] 12개 과목별 정답률 바 차트 표시
- [ ] 합격선 40% 미달 과목 빨간색 경고
- [ ] 앱 재시작 후 통계 유지
- [ ] 모바일 레이아웃 버튼 겹침 없음
- [ ] T-01 ~ T-10 전체 테스트 통과

디자인팀장 드림

UI-1 ~ UI-3 설계 완료. 조대표님 승인 후 노팀장 → 고팀장 전달 바랍니다.